

Prior-Boosted GRL: Microarchitecture Design Space Exploration via Graph Representation Learning

Zheng Wu¹, Jinyi Shen¹, Xiaoling Yi, Li Shang, *Member, IEEE*, Fan Yang¹, *Member, IEEE*,
and Xuan Zeng¹, *Senior Member, IEEE*

Abstract—The design space exploration (DSE) of contemporary microprocessors faces a significant challenge of high-computational cost. In this context, we introduce Prior-boosted graph representation learning (GRL), a novel framework for the DSE of the microarchitectures the microprocessors underpinned by graph embeddings. Using GRL, Prior-boosted GRL constructs a compact and continuous vector space for design representation. This framework is further boosted by an efficient sampling algorithm informed by prior knowledge, which is instrumental in generating a superior set of initial designs to accelerate the exploration process. A well-designed ensemble surrogate model is combined with the multiobjective Bayesian optimization to explore the design space holistically within this graph-embedding domain. Rigorous experimental evaluations conducted on the RISC-V Berkeley-Out-of-Order Machine (BOOM) platform demonstrate that Prior-boosted GRL substantially surpasses preceding methods, achieving a 107.79% enhancement in Pareto front quality compared to the state-of-the-art DSE algorithm. It also outstrips manual designs on performance, power, and area metrics. As of this writing, Prior-boosted GRL holds the first place in the ICCAD 2022 CAD Contest evaluation platform.

Index Terms—Bayesian optimization (BO), design space exploration (DSE), ensemble learning, graph representation learning (GRL), processor microarchitecture, RISC-V, sampling algorithm.

I. INTRODUCTION

TECHNOLOGY scaling and microarchitectural evolution are two drivers of the exponential growth in computing power. Over the past few decades, processor microarchitecture has undergone tremendous evolution. Many performance-enhancing microarchitectural features have been introduced,

such as pipelining, cache, branch predictor, and look-aside-buffer. Meanwhile, with the ever-increasing complexity and evaluation cost, microarchitecture design space exploration (DSE) has become increasingly challenging. Further, coupled with the increasing performance, power, and area (PPA) requirements and stringent time-to-market pressure, an automated approach to microarchitectural DSE is a must.

Efficient exploration of the high-dimensional design space is the primary challenge in microarchitecture DSE. On the one hand, the design of modern microprocessors requires the exploration of numerous design configurations, a combinatorial explosion in terms of microarchitecture design parameters. On the other hand, each configuration requires careful evaluation through costly simulations. Furthermore, design tradeoffs between negatively correlated goals require a thorough exploration of the multidimensional Pareto front, further complicating the exploration problem.

Manual DSE relies on the experiences of computer architects. However, this approach struggles with the scalability demands of new processors. It also lacks the adaptability needed for complex architectures and rapid advancements in modern microprocessor technology. Also, there are some techniques to accelerate processor performance evaluation. On the one hand, processor behavior-level simulators, such as GEM5 [1], [2], [3], are widely used for fast architecture exploration. However, these cycle-accurate simulators (CASs) have relatively lower-performance modeling accuracy compared to RTL-level circuit evaluation. On the other hand, the efficiency of performance simulations can be enhanced by employing various techniques, such as sampling methods, clustering algorithms [1], [4], [5], [6], and sensitivity analysis [7], to reduce benchmark simulation time. While these approaches effectively reduce simulation time for each design, they decrease the accuracy of processor simulation, potentially leading to suboptimal outcomes in the DSE process.

Statistical and machine learning-based methods have recently been introduced for efficient processor microarchitecture performance modeling [8], [9], [10], [11], [12]. These methods often treat the processor microarchitecture as a black box and leverage a training dataset to train a surrogate microarchitecture model to predict the processor's PPA metrics. Once trained, the surrogate model can support efficient performance prediction, e.g., thousands of predictions in a few seconds. Sitting the predictive model in the inner loop of the optimization flow provides a promising opportunity for fast microarchitecture DSE. However, this high-level modeling

Manuscript received 3 March 2024; revised 19 May 2024 and 30 July 2024; accepted 30 August 2024. Date of publication 10 September 2024; date of current version 21 February 2025. This work was supported by the National Natural Science Foundation of China (NSFC) Research Project under Grant 92373207. This article was recommended by Associate Editor W. Liu. (Zheng Wu and Xiaoling Yi contributed equally to this work.) (Corresponding author: Fan Yang.)

Zheng Wu, Jinyi Shen, Fan Yang, and Xuan Zeng are with the State Key Laboratory of Integrated Circuits and Systems and the School of Microelectronics, Fudan University, Shanghai 200433, China (e-mail: yangfan@fudan.edu.cn).

Xiaoling Yi was with the State Key Laboratory of Integrated Circuits and Systems, Fudan University, Shanghai 200433, China. She is now with the Department of Electrical Engineering—Microelectronics Circuits and Systems, KU Leuven, 3000 Leuven, Belgium.

Li Shang with the State Key Laboratory of Integrated Circuits and Systems and the School of Computer Science, Fudan University, Shanghai 200433, China.

Digital Object Identifier 10.1109/TCAD.2024.3457376

approach requires a large amount of training data to ensure sufficient inference accuracy, which needs costly simulations to obtain. For instance, İpek et al. [8] requires 2.5K training data to build a predictive model for multiprocessors. Since the surrogate model lacks prior knowledge of the microprocessor behavior, it learns from scratch.

In this research, we emphasize the critical role of embedding prior knowledge of microarchitecture, including microarchitecture organization and parameter dependencies, enabling more efficient microarchitecture DSE. Designing microprocessor architecture is a highly structured process, constrained by a series of microarchitecture-specific dependencies, such as data and control correlations of the microprocessor pipeline structure. For example, the instruction *decode width* must divide the *reorder buffer entry number* and the instruction *fetch width* should be larger than the instruction *decode width*. These inherent characteristics and design constraints of microarchitecture significantly lessen the cost of exploration.

A previous study [13] in the realm of multiprocessor DSE leveraged these microarchitecture details to set up the exploration as a discrete-space *Markov decision process*. It avoids unnecessary simulations of certain design points, providing more than one order of magnitude reduction in the number of simulations needed for exploration. However, such domain knowledge is heuristic, making it difficult to formulate and capture. Recently, Berkeley-Out-of-Order Machine (BOOM)-Explorer [14] introduced a sampling algorithm that segments the entire design space based on key microarchitectural parameters. It then selects representative design points in each segment to reduce the initial sample size needed for training surrogate models. However, the sampling designs of this method are relatively divergent, lacking focused targeting of the optimal point. This results in the poor modeling accuracy of the surrogate model near the optimal point and low efficiency of the optimization process. Despite these challenges, these works demonstrate the significant potential of incorporating domain knowledge into the DSE process.

To overcome the challenges in DSE, this article presents *Prior-boosted* graph representation learning (GRL), an automated microarchitecture DSE framework, which is equipped with architecture graph learning representation, a prior knowledge-based initial sampling method, and an efficient multiobjective Bayesian optimization (BO) framework. A preliminary version of Prior-boosted GRL was published in [15]. The main contributions of this article are summarized below.

- 1) We introduce a new directed acyclic graph (DAG) model for representing processor microarchitecture, processed through a graph neural network (GNN) to extract key features as embeddings in a graph embedding space. This approach, a first in the field, uses GRL to capture the intrinsic characteristics of microprocessors, facilitating efficient DSE.
- 2) We propose PriorBoost, a prior knowledge-based sampling method, to construct a high-quality initial design set. In the PriorBoost algorithm, designs with extreme and medium parameter values are selected for initial sampling due to their high likelihood of being Pareto-optimal points. This approach significantly enhances the

efficiency of the optimization process and outperforms existing initial sampling algorithms.

- 3) We propose a well-designed ensemble model-based black-box optimizer, which uses the expected hypervolume improvement (EHVI) acquisition function for efficient DSE. This method exhibits strong robustness and effectively explores high-quality designs that strike a good balance among PPA.
- 4) We comprehensively validate our Prior-boosted GRL framework on a configurable RISC-V processor, BOOM. Experimental results indicate that, compared to the current state-of-the-art DSE algorithms, Prior-boosted GRL can enhance the golden distance metrics of the searched designs by 107.79%. The iteration rounds for processor design exploration are reduced by 57.5% to achieve the same design quality, and the obtained results surpass some manually designed outcomes. In the ICCAD CAD Contest 2022 ranking system, the comprehensive performance of Prior-boosted GRL currently holds the first position.

The remainder of this article is organized as follows. Section II introduces the BOOM processor and defines the microarchitecture DSE problem. Section III presents Prior-boosted GRL, the proposed automated microarchitecture DSE framework. Section IV demonstrates the experiment results. Section V concludes the work.

II. PRELIMINARIES

A. Design Space of BOOM Microarchitecture

BOOM [16], which is an open-source RISC-V out-of-order microarchitecture, is popular in architecture research for its competitive power efficiency. BOOM consists of ten pipeline stages, which can be further categorized into three functional modules: 1) frontend; 2) execute; and 3) load/store units. Microarchitecture is the implementation of instruction set architecture (ISA). There are numerous microarchitecture design parameters associated with the three modules of BOOM, including pipeline width, cache size, reorder buffer entries, and load-store queue length. These design parameters have a direct impact on the PPA of the processor.

ICCAD Contest 2022 Problem C [17] focuses on the BOOM microarchitecture DSE problem, in which 29 microarchitecture parameters are chosen. The names and candidate values of the parameters are listed in Table I. Due to certain parameter constraints in the BOOM processor design, the search space does not encompass all possible combinations of these parameter options. For example, *FetchWidth* should not be smaller than *DecodeWidth*, *ROBEntries* should be divisible by *DecodeWidth*, etc. Overall, the ICCAD Contest 2022 provides a microarchitecture design space comprising 1.5×10^4 design candidates. The design space is divided into five partitions according to different *DecodeWidth* values, which are critical parameters affecting the PPA of the BOOM microprocessor. Each partition is a design subspace in this problem. With Chisel [18], a variety of BOOM cores with different microarchitecture configurations can be generated. Conducting RTL-level simulations for a BOOM design on an

Intel Xeon Platinum 8354H CPU@3.10-GHz cluster requires 3–6 h, making a comprehensive exploration of the entire search space a staggering 7.7-year endeavor.

B. Graph Learning Techniques

1) *Graph Learning*: Graph learning is a subfield of machine learning that focuses on the analysis and interpretation of data structured in graphs. In these graphs, entities are represented as nodes, and their relationships are depicted as edges. This approach is particularly useful for understanding complex relational data.

One of the primary techniques in graph learning is graph embedding [19], which involves mapping nodes of a graph to vectors in a lower-dimensional space. This representation allows for the quantification of node similarities and the preservation of graph topology in a way that is amenable to further machine learning tasks.

Another significant technique is GNNs [20], [21]. GNNs are specialized neural network architectures designed to handle graph-structured data. They work by aggregating information from a node's neighbors to learn powerful representations for nodes or entire graphs. This enables the application of machine learning models directly on graph data, facilitating tasks, such as node classification and graph property prediction [22].

Variational Graph Autoencoder (VGAE): VGAEs [23] have been successfully applied to extracting meaningful features and learning interpretable latent representations from graph-structured data. Let $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ denote an undirected graph with $N = |\mathcal{V}|$ nodes. $\mathbf{A}$ is the adjacency matrix and $\mathbf{X}$ is an $N \times D$ feature matrix of $\mathcal{G}$. $\mathbf{Z}$ is an $N \times F$ feature matrix of learned latent representations.

The VGAE encoder uses a two-layer graph convolution network (GCN)

$$q(\mathbf{Z}|\mathbf{X}, \mathbf{A}) = \prod_{i=1}^N \mathcal{N}(\mathbf{z}_i | \boldsymbol{\mu}_i, \text{diag}(\boldsymbol{\sigma}_i^2))$$

where $\boldsymbol{\mu} = \text{GCN}_{\boldsymbol{\mu}}(\mathbf{X}, \mathbf{A})$, $\log \boldsymbol{\sigma} = \text{GCN}_{\boldsymbol{\sigma}}(\mathbf{X}, \mathbf{A})$. The VGAE decoder takes the form of inner product between latent variables, $p(\mathbf{A}|\mathbf{Z}) = \prod_{i=1}^N \prod_{j=1}^N \text{sigmoid}(\mathbf{z}_i^\top \mathbf{z}_j)$, where $\text{sigmoid}(\cdot)$ is the logistic sigmoid function.

VGAE encoder and decoder are jointly learned by optimizing the variational lower-bound $\mathcal{L}$

$$\mathcal{L} = \mathbb{E}_{q(\mathbf{Z}|\mathbf{X}, \mathbf{A})} \log[p(\mathbf{A}|\mathbf{Z})] - \text{KL}[q(\mathbf{Z}|\mathbf{X}, \mathbf{A})||p(\mathbf{Z})] \quad (1)$$

where $\text{KL}[p(\cdot)||q(\cdot)]$ is the Kullback–Leibler divergence between $p(\cdot)$ and $q(\cdot)$, and $p(\mathbf{Z})$ is a Gaussian prior.

However, VGAE falls short in handling DAGs due to its inability to capture directional relationships and its limited graph generation abilities. DAG variational autoencoder (DVAE) [24], on the other hand, provides significant improvements in modeling, generating, and processing DAGs, making it a more suitable choice for applications involving DAG structures.

DVAE is a powerful generative model comprising two neural networks: 1) an encoder that transforms a DAG into a low-dimensional latent representation and 2) a decoder that

reconstructs a DAG. The encoder and decoder are trained together by maximizing the evidence lower bound.

VGAEs and DVAEs perform unsupervised learning to generate node embeddings from graph structures. The unsupervised learning process primarily revolves around learning representations of nodes in a graph without relying on labeled data. The training data is obtained from the structure of the graph itself, which consists of nodes and edges.

VGAEs and DVAEs perform unsupervised learning by using the graph's structure and node features as training data, which are inherently unlabeled. The process begins with representing the graph using its adjacency matrix $\mathbf{A}$ and node features $\mathbf{X}$. The encoder, typically a GCN, processes this input to produce latent variables $\mathbf{Z}$, which are sampled from a Gaussian distribution defined by the encoder. The decoder then reconstructs the graph structure by calculating the inner product of these latent variables to estimate the probability of edges between nodes. The training objective combines the reconstruction loss, which measures the difference between the original and reconstructed adjacency matrices, and the KL-divergence loss to align the learned distribution with a prior distribution. Through this process, VGAEs learn meaningful node embeddings without the need for labeled data.

C. Sampling Algorithm

Sampling algorithms are fundamental techniques in statistics and machine learning, used to draw representative subsets from larger datasets or distributions. These algorithms enable efficient approximation of complex models and facilitate tasks, such as Monte Carlo simulations, Bayesian inference, and stochastic optimization. Key methods include simple random sampling, stratified sampling [25], and importance sampling [26], each serving different purposes based on the problem at hand. In machine learning, sampling is crucial for training models on large datasets, enabling techniques like mini-batch gradient descent [27], where random subsets of data are used to update model parameters iteratively. Advanced sampling methods, such as Markov chain Monte Carlo (MCMC) [28] and Gibbs sampling [29], are essential for estimating posterior distributions in Bayesian models, making them indispensable tools for probabilistic reasoning and inference.

D. Bayesian Optimization

BO [30] is a powerful strategy for optimizing black-box functions that are expensive to evaluate. It utilizes a probabilistic model, known as the surrogate model, to approximate the objective function. This model predicts the function's outcomes and quantifies uncertainty in areas of the search space where data is sparse. Acquisition functions then guide the optimization by balancing the exploration of new areas against exploiting known good areas. The initial sampling phase, typically using simple random or grid sampling, establishes a baseline by scattering points throughout the search space. Collectively, these components streamline the search for optimal solutions in complex, high-cost scenarios.

TABLE I
DESIGN SPACE OF BOOM MICROARCHITECTURE IN ICCAD CONTEST 2022

Modules	Parameters	Candidate values	Descriptions
Fetch	FetchWidth	4, 8	Number of fetched instructions once
Decoder	DecodeWidth	1, 2, 3, 4, 5	Number of decode instructions once
ISU	MemDispatchWidth	1, 2, 3, 4, 5	Number of dispatched memory instructions once
	MemIssueWidth	1, 2	Number of issued memory instructions once
	MemNumEntries	6, 8, 10, 12, 14, 16, 20, 22, 24, 26, 28	Entries of memory instructions
	IntDispatchWidth	1, 2, 3, 4, 5	Number of dispatched integer instructions once
	IntIssueWidth	1, 2, 3, 4, 5	Number of issued integer instructions once
	IntNumEntries	6, 8, 12, 20, 24, 32, 36, 40, 44, 48	Entries of integer instructions
	FpDispatchWidth	1, 2, 3, 4, 5	Number of dispatched float point instructions once
	FpIssueWidth	1, 2, 3, 4, 5	Number of issued float point instructions once
IFU	FpNumEntries	6, 8, 12, 20, 24, 28, 32, 36	Entries of float point instructions
	BranchTag	6, 8, 10, 12, 14, 16, 18, 20, 22, 24, 26	Size of branch tag
	FetchBufferEntries	6, 8, 12, 14, 16, 20, 21, 24, 30, 32, 35, 36, 40, 45	Entries of the fetch buffer register
ROB	FetchTargetQueue	14, 16, 20, 30, 32, 36, 40, 44, 48, 50	Length of Fetch Target Queue
	ROBEntries	30, 32, 34, 36, 60, 64, 72, 90, 96, 108, 120, 128, 130, 132, 136, 140	Entries of the reorder buffer
PRF	INTPhysRegisters	42, 52, 62, 70, 80, 90, 100, 110, 118, 128, 138, 146	Number of physical integer registers
	FPPPhysRegisters	38, 48, 54, 58, 64, 74, 86, 96, 106, 118, 128, 138, 146	Number of physical float point registers
LSU	LDQEntries	6, 8, 12, 16, 20, 24, 28, 32, 36	Entries of the loading queue
	STQEntries	6, 8, 12, 16, 20, 24, 28, 32, 36	Entries of the store queue
I-Cache/MMU	Sets	32, 64	Sets of L1 I-Cache
	Ways	1, 4, 8	Ways of L1 I-Cache
	I-TLBSets	1, 2	Sets of Table Look-aside (TLB) in L1 I-Cache
	I-TLBWays	16, 32	Ways of TLB in L1 I-Cache
D-Cache/MMU	Sets	64	Sets of L1 D-Cache
	Ways	2, 4, 8	Ways of L1 D-Cache
	ReplacementPolicy	0, 1	L1 Data Cache replacement policy choice
	MSHR	2, 4, 8	Entries of Miss Status Handling Register
	D-TLBSets	1, 2	Sets of TLB in L1 D-Cache
	D-TLBWays	8, 32	Ways of TLB in L1 D-Cache

E. Design Space Exploration Problem Formulation

Microarchitecture DSE aims to discover superior designs that achieve an optimized tradeoff among PPA values. As such, the microarchitecture DSE problem can be formulated as a multiobjective optimization problem, as follows:

$$\begin{aligned} &\text{minimize } \mathbf{f}(\mathbf{x}) = \{-\text{Perf}(\mathbf{x}), \text{Power}(\mathbf{x}), \text{Area}(\mathbf{x})\} \\ &\text{s.t. } \mathbf{x} \in \mathbf{D} \end{aligned} \quad (2)$$

where $\mathbf{f}(\mathbf{x})$ denotes the objective function of DSE. $\text{Perf}(\mathbf{x})$, $\text{Power}(\mathbf{x})$, and $\text{Area}(\mathbf{x})$ represent the PPA of the design with parameter $\mathbf{x}$, respectively. $\mathbf{D}$ denotes the whole design space.

In multiobjective optimization, the goal is not to find a singular solution point but rather a set of Pareto-optimal solutions.

Definition 1 (Dominate): For a minimization problem with n objectives, $\mathbf{x}_1$ dominates $\mathbf{x}_2$ if

$$\begin{cases} \forall i \in [1, n], f_i(\mathbf{x}_1) \leq f_i(\mathbf{x}_2) \\ \exists j \in [1, n], f_j(\mathbf{x}_1) < f_j(\mathbf{x}_2) \end{cases} \quad (3)$$

where $f_i(\mathbf{x})$ is the i th objective function. $\mathbf{x}_1 \succ \mathbf{x}_2$ denotes that $\mathbf{x}_1$ dominates $\mathbf{x}_2$, which means $\mathbf{x}_1$ is a superior solution to $\mathbf{x}_2$ in the context of a multiobjective minimization problem. A solution point $\mathbf{x}_i$ is nondominated if it is not dominated by any other solution point $\mathbf{x}_j$.

Definition 2 (Pareto Optimality): For multiobjective optimization, the Pareto optimality set is comprised of a collection of nondominated points. This set represents the solution to the multiobjective problem, where each point in the set offers a unique tradeoff between the objectives.

The goal of microarchitecture DSE is to find the Pareto optimality configuration parameter set.

III. ALGORITHM

A. Overview

This article presents Prior-boosted GRL, an automated microarchitecture DSE framework equipped with architecture graph learning representation, a prior knowledge-based initial sampling algorithm, and an efficient multiobjective BO framework.

The Prior-boosted GRL framework is shown as depicted in Fig. 1, illustrating the workflow of the processor architecture exploration method. The Graph Learning ① transforms the design space into a graph embedding space for the BO framework ③ to search. The Prior Sampling Algorithm ② provides a set of high-quality designs as the starting point of the BO ③. First, we construct a DAG representation of the processor design based on the pipeline structure to encode the microarchitecture interdependence. Then, we map the discrete graph design space to a continuous graph embedding space using a customized GNN trained with the DVAE framework. The graph embedding space serves as the search space of the BO. Subsequently, based on prior knowledge of architectural exploration, we transform the cold start of exploration into two multiobjective optimization problems for constrained parameters, thereby creating a high-quality initial design data set. The initial sampled designs are encoded by GNN, serving as a starting point for BO. Efficient exploration is carried out in the microarchitecture graph design space using a multiobjective BO algorithm equipped with ensemble learning and the EHVI acquisition function. The BO algorithm is an iterative optimization method that employs an ensemble surrogate model to learn the relationship between the microarchitecture graph embedding and PPA. It then utilizes the EHVI acquisition function to guide the optimization by balancing the exploration of new areas against exploiting known good areas. Suggested designs of the EHVI function are simulated using

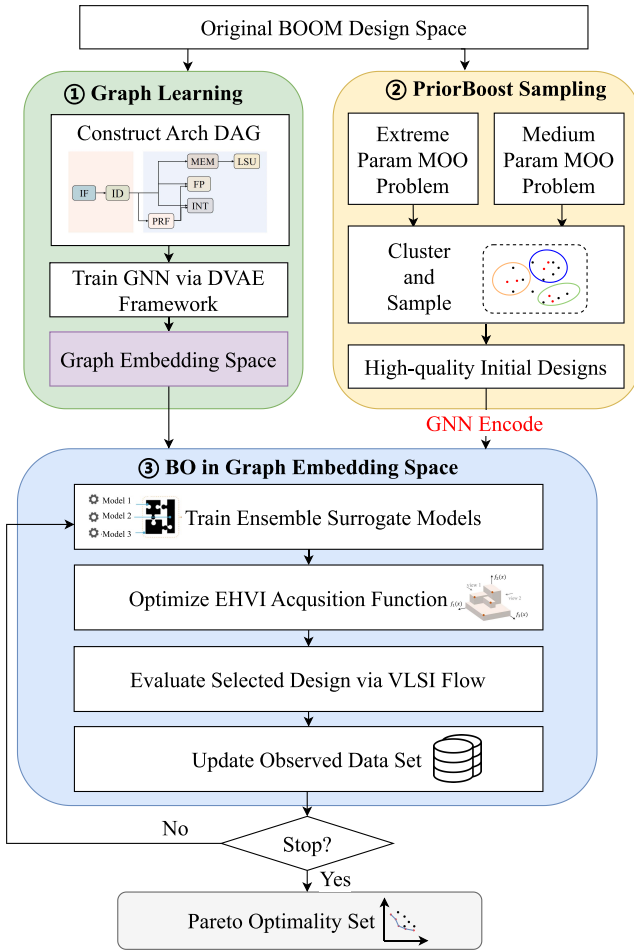


Fig. 1. Prior-boosted GRL framework overview.

RTL-level simulation tools. The simulation results are added to the observed data set along with the microarchitecture designs to assist in modeling with the ensemble surrogate model and for the next round of exploration. After a predetermined number of N optimization iterations, Pareto-optimal designs are extracted from the observed dataset as the result of the DSE framework.

B. Microarchitecture Graph Representation Learning

1) *Motivation and Overview of Graph Learning*: The backbone pipeline of the BOOM processor is divided into three microarchitecture stages: 1) FrontEnd; 2) execute; and 3) load store unit (LSU). In the FrontEnd stage, there are Fetch and Decode Units, while in the Execution Stage, there are Integer, Floating Point, Memory Execution Units, and Physical Register Units. The instructions are fetched and decoded in the FrontEnd stage. These decoded instructions are then sent to the Execute stage for processing. Finally, the execution results are passed to the LSU for storage. Due to the intrinsic pipeline structure of the processor, there are certain producer-consumer relationships among its various components. For example, in the FrontEnd stage, the instruction cache, fetch unit, and fetch buffer (producer) provide fetched instructions to the decode unit (consumer) for decoding. The data from the physical

TABLE II
MICROARCHITECTURE PARAMETERS EMBEDDED AS NODE FEATURES

Node	Node feature
IF	FetchWidth, FetchBufferEntries, BranchTag, FetchTagetQueueEntry, ICacheSets, ICacheWays, ICacheTLBSets, ICacheTLBWays
ID	DecodeWidth, ROBEntries
PRF	IntPhysRegisters, FpPhysRegisters
INT	IntDispatchWidth, IntIssueWidth, IntNumEntries
FP	FpDispatchWidth, FpIssueWidth, FpNumEntries
MEM	MemDispatchWidth, MemIssueWidth, MemNumEntries
LSU	LDQEntries, STQEntries, DCacheSets, DCacheWays, ReplacementPolicy, MSHR, DCacheTLBSets, DCacheTLBWays

register (producer) is handed over to the integer, floating-point, and memory units (consumer) for execution.

Intuitively, to minimize pipeline stalls, the throughput of downstream components should match that of the upstream components correspondingly. This introduces microarchitecture dependencies between downstream and upstream components, which are represented by microarchitectural parameters. For instance, *FetchBufferEntry* must be larger than *FetchWidth*, and the *DecodeWidth* should equal to or smaller than *FetchWidth*, whereas *DecodeWidth* should divide *RobEntries* and *FetchBufferEntry*. This interdependence between the producer and consumer components can be leveraged to facilitate the optimization process of the microarchitecture [31].

In traditional black-box optimization algorithms [8], [9], [14], microarchitecture parameters are treated as a set of parameter vectors for optimization. These approaches overlook the interdependence between processor microarchitectures and fail to incorporate additional useful information.

To model the interdependence, we introduce graph learning techniques. We propose a processor pipeline-based microarchitecture graph to model processor microarchitecture, capturing key microarchitectural interdependence. Using GNN, we map this microarchitecture graph into a latent graph embedding space. Compared to simple black-box optimization methods, our graph learning technique encodes the dependencies between microarchitecture components into the latent space, creating a higher-quality search space and enhancing optimization results, as will be discussed in the experimental results section.

2) *Microarchitecture Graph Model*: In our proposed Prior-boosted GRL framework, the structure of an out-of-order superscalar microprocessor is modeled as a DAG to capture the microarchitectural interdependencies. Specifically, as shown in Fig. 2, using DAG, the superscalar microarchitecture is modeled as two subgraphs, namely, front end and execution core. The front end (producer), consisting of two graph nodes instruction fetch (IF) and instruction decode (ID), models IF, decoding, and dispatch. The Execution Core (consumer) consists of five graph nodes, physical register file (PRF), INT (integer-related instruction execution units), FP (floating point-related instruction execution units), MEM (memory-related instruction execution units), and LSU, orchestrating the execution of the translated microoperations and results write-back.

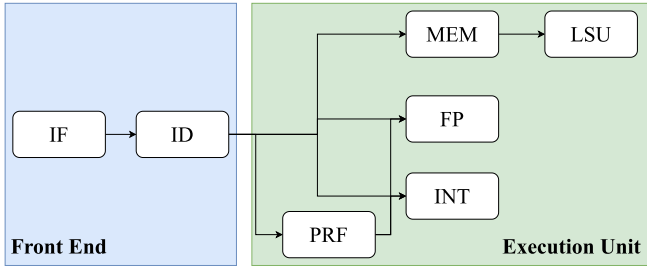


Fig. 2. Microarchitecture interdependence DAG of out-of-order superscalar microprocessors.

In the microarchitecture DAG, nodes represent key microarchitecture components, with key parameters embedded as node features, indicating the size of the components. Table II enumerates the microarchitecture features of the BOOM processor encoded in each graph node. On the other hand, each directed edge represents a specific producer-consumer relationship between microarchitecture components based on the processor instruction flow. For example, the edge from *IF* to *ID* indicates instructions flowing from the fetch phase to the decode phase. The directed edges also encode parameter interdependence relationships. The edges from *ID* to *INT*, *FP*, and *MEM* execution units capture the interdependence between the execution unit and the decode unit.

In summary, the proposed DAG contains a rich set of microarchitecture information, such as microarchitecture configurations, intercomponent data, and control dependencies as well as parameter correlations. Representing microarchitectural parameters as a DAG can increase the amount of information available for optimization, thereby enhancing the effectiveness of the optimization process. The microarchitecture DAG is applicable to common superscalar processors and can be further extended to incorporate additional microarchitecture features or support other disparate processors.

3) *Customized Graph Neural Network Model*: The microarchitecture DAG is in a discrete representation space and is difficult to be directly optimized. Inspired by advancements in GNN research [20], [24], we introduce a customized GNN model to convert the microarchitecture DAG into a continuous graph embedding space. This transformation facilitates more efficient exploration of the design space, enabling a streamlined approach to microarchitecture optimization.

Algorithm 1 summarizes the proposed GNN-based GRL process. It uses asynchronous information propagation to encode the computations on the microarchitecture DAG. Specifically, the graph nodes are arranged according to the topological order of the directed edges, and the hidden states of the nodes are updated accordingly, which conforms to microprocessor instruction execution flow and parameter dependencies.

Diverging from the methods in [20], our GNN approach considers both node and edge features concurrently, providing a holistic analysis. As demonstrated in Algorithm 1, we employ the update function $\mathcal{U}$ and the aggregation function $\mathcal{A}$, similar to standard graph convolutional networks, to determine

Algorithm 1 Graph Embedding Algorithm in Prior-Boosted GRL

```

1: INPUT: DAG represents the microarchitecture parameters  $G = (V, E)$ ;
2: OUTPUT: Embed feature  $z$  of  $G$  in continuous space;
3: Encode:
4: for  $v$  in  $V$  do
5:   get  $v$ 's predecessors  $V_p$ ;
6:    $h_v^{in} = \mathcal{A}(h_{V_p}: V_p \rightarrow v)$ ;
7:    $h_v = \mathcal{U}(V_v, h_v^{in})$ ;
8: end for
9:  $q(Z|G) = MLP_{encode}(h_v)$ ;
10: sample  $z$  from  $q(Z|G)$ ;
11: return  $z$ ;

```

the hidden state h_v of each graph node. For each node v in the DAG, h_v^{in} is the aggregation of the states of all v 's predecessors. Fig. 3 ① illustrates how the instruction execution and internode parameter dependencies PF are propagated. Since nodes have different numbers of features, a front-end linear transformation layer is introduced to ensure the same node feature size across hidden state updates and aggregation. Functions $\mathcal{A}$ and $\mathcal{U}$ are described as follows:

$$\mathcal{A}(h_p; v_p \rightarrow v) = \sum_{v_p} g(V'_p \times h_p) \odot m(V'_p \times h_p) \quad (4)$$

$$\mathcal{U}(V_v, h_v^{in}) = \text{GRU}(W_v V_v, h_v^{in}) \quad (5)$$

where g and m serve as a gating network and a mapping network, respectively. $\odot$ indicates the Hadamard product. V_v represents the feature of node v , while V_p denotes the node feature of its predecessors v_p . $V'_p = W_p V_p$, where W_p is the linear transformation layer of v_p . The update function $\mathcal{U}$ employs a gated recurrent unit (GRU) [32].

As depicted in Fig. 3 ②, the DAG incorporates a virtual input node and an output node, establishing the initial and terminal points of the graph. The virtual output node succeeds the INT, FP, and LSU nodes, concluding the sequence of information propagation within the microarchitecture DAG. The information propagation in the microarchitecture DAG initiates at the virtual input node V_{in} and ends at the virtual output node V_{out} . The hidden state at the output node $h_{V_{out}}$ can be regarded as a microarchitecture representation that extracts the key processor microarchitecture information. This abstracted representation is then channeled through two multilayer perceptrons (MLPs) to derive the graph embedding vector z .

4) *DVAE Training Framework*: Fig. 3 illustrates our proposed graph learning scheme based on the DVAE [24]. Based on the DAG that captures the microarchitectural interdependence, the microarchitectural design space is then represented as a graph space. Subsequently, the customized GNN serves as the encoder for the DVAE, which maps the microarchitectural DAGs into a continuous graph embedding space. The DVAE is trained using the variational lower-bound function to optimize the encoder and decoder parameters.

The training of the DVAE model is unsupervised. Only the microarchitecture design space is needed for the training

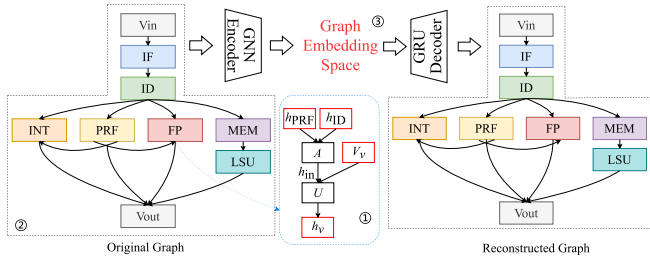


Fig. 3. Microarchitecture graph learning framework based on the DVAE technique.

process. In our proposed framework, we represent microarchitecture designs as Microarchitecture DAGs. By encoding each design in the design space as a DAG, we generate a set of DAGs representing the microarchitecture design space. This set forms our unlabeled training set for the DVAE.

The training approach for the customized GNN follows the DVAE framework, as depicted in Fig. 3 ③. The training process minimizes the discrepancies between the input and output DAGs by maximizing the variational low-bound L concerning the variational parameters W_i . The lower bound is defined as

$$\mathcal{L}(\phi, \theta; X) = \mathbb{E}_{z \sim q_\theta(Z|X)} [\log p_\phi(X|Z)] - \text{KL}[q_\theta(Z|X) \| p(Z)] \quad (6)$$

where $p_\phi(X|Z)$ denotes a probabilistic generative model (the decoder) and $q_\theta(Z|X)$ represents an approximated posterior distribution (the encoder). Here, ϕ and θ are the parameters of the decoder and encoder, respectively. The term $\text{KL}[q(\cdot) \| p(\cdot)]$ signifies the Kullback–Leibler divergence between $q(\cdot)$ and $p(\cdot)$. We take a Gaussian prior $p(Z) = \prod_i p(z_i) = \prod_i \mathcal{N}(0, I)$. This ensures that the graph embedding vector z follows the Gaussian distribution.

In the DVAE’s decoder, we utilize a $\text{GRU}_{\text{decode}}$ to update the embedded vector, which is subsequently fed into an $\text{MLP}_{\text{decode}}$. This MLP computes the node feature probability $P_{\text{node feature}}$. Based on the $P_{\text{node feature}}$, the reconstructed node V_{re} is sampled, leading to the formation of the reconstructed DAG $G_{\text{re}} = (V_{\text{re}}, E)$.

The objective of training a DVAE is to identify the GNN network parameters that can accurately map a DAG to a graph embedding. Training a GNN model to map the search space in Table I to the graph embedding space takes roughly 10 h on a GeForce RTX 2080 Ti GPU. Additionally, for one specific architecture, we only need to train the D-VAE once.

Our method can be adapted to other processor architectures. For different processor architectures, it is necessary to construct distinct architecture DAGs. The microarchitecture DAG is designed at the module level of granularity. If the overall modular structure of the processor remains unchanged, the structure of the DAG does not need to be altered. If there are different microarchitecture parameters, the feature vectors of the nodes in the DAG and the size of the linear transformation layer in the GNN need to be modified. However, the modeling approach and algorithmic process we propose are general and applicable to various scenarios.

C. PriorBoost—Efficient Initial Sampling Strategy

A high-quality initial sampling set serves as a favorable starting point, which often leads to superior optimization outcomes and reduces runtime. Even though the standard black-box BO flow typically assumes no prior knowledge regarding the optimization problem, expert knowledge should be leveraged whenever possible to improve the quality of the initial samples, enabling a more efficient optimization and better-decision-making process.

1) *Extreme Sampling Method*: We notice that the design parameters are bounded during DSE. The lower and upper bounds of the design parameters often correspond to extreme PPA values. For instance, increasing the buffer size improves performance but also increases chip area. Additionally, reducing the *DecodeWidth* lowers performance but also reduces power and area. These observations suggest that the lower and upper bounds of the design parameters have a high likelihood of representing extreme PPA metrics which is in the Pareto-optimal set. However, in reality, the parameters cannot reach the lower/upper bounds simultaneously due to the design constraints of the processor. Therefore, we consider the minimization and maximization of parameters under the given constraints as extreme configurations. This procedure can be formulated as two multiobjective optimization problems as follows:

$$\begin{aligned} &\text{minimize } \mathbf{x} = \{x_1, x_2, \dots, x_N\} \\ &\text{w.r.t. constraints of parameters} \end{aligned} \quad (7)$$

$$\begin{aligned} &\text{maximize } \mathbf{x} = \{x_1, x_2, \dots, x_N\} \\ &\text{w.r.t. constraints of parameters} \end{aligned} \quad (8)$$

where $\mathbf{x}$ is the parameter vector. Note that the multiobjective optimization problems (7) and (8) can be solved efficiently without extensive evaluation of the PPA of the designs. The focus is on considering the constraints of the parameters to determine the Pareto-optimal points for each problem. This approach allows for efficient exploration of the design space without the need for extensive evaluations of the PPA metrics. Based on the definitions of dominance and Pareto optimality in Section II-E, we can obtain solutions to both problems from each design subspace, which has 25 extreme design configurations.

2) *Medium Sampling Method*: The proposed extreme sampling algorithm generates samples that can either maximize performance with high-power consumption and large area or minimize performance with low-power consumption and small area. However, it is possible that these extreme samples often concentrate at the two ends of the Pareto frontier in the PPA space. In other words, they are biased toward either the highest performance or the lowest-power consumption and area. Such biased distribution results in limited coverage of the central regions of the Pareto frontier, where solutions shall exhibit more balanced tradeoffs in terms of PPA.

We observe that design parameters close to the medium region frequently result in balanced PPA values. This observation is utilized to sample medium design configurations to enable the coverage of the central regions near the Pareto frontier in the initial sampling stage. We calculate the medium

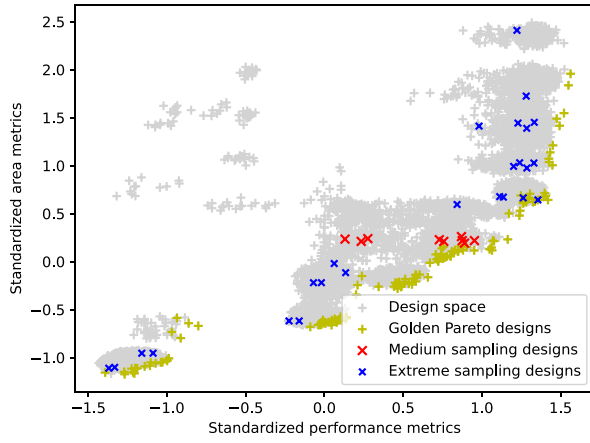


Fig. 4. Distributions of the candidate samples from extreme sampling and medium sampling in PPA space.

value for each parameter by determining the median of the parameter's candidate values. Our objective is to align the parameter values as closely as possible to their respective medium values, taking into account the design constraints of the microprocessor. This task can be formulated as the following multiobjective optimization problem:

$$\begin{aligned} &\text{minimize} \quad \mathbf{d} = \{d_1, d_2, \dots, d_N\} \\ &\text{where } d_i = |x_i - m_i| \\ &\text{w.r.t} \quad \text{constraints of parameters} \end{aligned} \quad (9)$$

where m_i denotes the medium value of the i th parameter. d_i represents the distance between the i th parameter value and its medium value. The *DecodeWidth* parameter value mirrors its median in the 3rd design subspace, where the designs tend to exhibit moderate PPA values. Thus, we address this multiobjective optimization problem within the 3rd design subspace, yielding nine samples that strike a balance among PPA values.

3) *Clustering and Sampling From Candidates*: The microarchitecture designs obtained from extreme sampling and medium sampling are merged to form the initial candidate sampling set. The distribution of these candidate designs in the standard performance-power space and performance-area space is depicted in Fig. 4. The figure illustrates that the extreme sampling points mainly cluster at the corners of the Pareto frontier in the PPA space, while the medium sampling points are predominantly located in the central region of the Pareto frontier. This observation further validates the prior knowledge regarding the relationship between parameter selection and PPA values.

It is known that evaluating a large number of designs can introduce significant overhead to the DSE process. Moreover, some points in the candidate sample set may be close to each other, providing limited value for the surrogate model to learn the features of the Pareto front. To address these concerns and achieve efficient and uniform sampling across the search space, we apply the k -means clustering algorithm [33] to cluster the candidate points. By partitioning and randomly selecting one point from each cluster, ten designs are selected to form the initial sample set, which balances the diversity and evaluation cost.

Based on prior knowledge, we construct three multiobjective optimization problems to sample high-quality designs near the Pareto frontier without time-consuming design evaluations. In Fig. 4, the sampled points span the region adjacent to the Pareto front from one end to the other. These sampled points serve as a high-quality starting point for the surrogate model to capture the features of the Pareto frontier and thus accelerate the optimization procedure of DSE.

D. Surrogate Model Ensemble

The surrogate model plays a critical role in BO by approximating the underlying latent function. Different surrogate models, such as random forests (RFs) [34], Gaussian processes (GPs) [35], deep ensemble models [36], and their derivatives, are commonly used in BO. Every machine learning model has its strengths and weaknesses, making each suited to specific tasks and challenges.

For black-box optimization problems with limited prior knowledge, *ensemble models* have proven to be effective in enhancing fitting performance and model robustness. For microarchitecture DSE, where the cost of training models is considerably lower than the evaluation cost, training multiple models becomes negligible in terms of overall computational cost. Therefore, utilizing ensemble models during microarchitecture DSE is beneficial as it improves the performance and reliability of the surrogate model without significantly increasing the computational burden.

1) *Base Model Selection*: To assess the performance of different surrogate models, we conduct cross-validation (CV) and hold-out validation (HV) experiments using data obtained from simulations of the BOOM microarchitecture. In these experiments, we evaluate the models' performances using 10 and 30 randomly selected data points, respectively. These experiments allow us to compare and analyze the effectiveness of various surrogate models in capturing the features of the microarchitecture graph learning design space.

Table III shows the average coefficient of determination (R^2) values on PPA data for different surrogate models. Among them, GP1 and GP2 represent GP [35] with Marten kernel and arc sine kernel, respectively. Two sparse variational GP (SVGP) [37] models are included. Additionally, various deep ensemble models incorporate different techniques, such as preconditioned stochastic gradient Langevin dynamics (PSGLDs) [39], Monte Carlo batch normalization (MCBN) [40], feature selection using stochastic gates (FE) [41], and Gumbel-Softmax (Gumbel) [42]. The bolded entries in Table III indicate favorable results. Based on the results in the table, GP2, sparse variational inference with deep Kernel learning (SVIDKL) [38], and RF model [34] exhibit outstanding performance in both CV and HV experiments. As a result, these models are selected as the base models for the surrogate ensemble model.

2) *Ensemble Model*: Given a training set with inputs $\mathbf{X} = \{\mathbf{x}_1, \dots, \mathbf{x}_n\}$ and corresponding outputs $\mathbf{y} = \{y_1, \dots, y_n\}$. Further, let's assume that we are making a prediction at a test point $\mathbf{x}_*$. The mean and variance formulas for the GP model

TABLE III
EXPERIMENT RESULTS OF CV AND HV FOR DIFFERENT MODELS

	GPI [36]	GP2	SVGP [38]	SVIDKL [39]	RF [35]	Deep Ensemble [37]	PSGLD [40]	MCBN [41]	FE [42]	Gumbel [43]
CV with 10 data	0.372	0.005	0.141	0.325	0.456	0.312	-1547.881	-0.267	0.828	0.675
CV with 30 data	0.858	0.915	0.879	0.910	0.837	0.215	-8.503	-2.081	0.849	0.470
HV with 10 data	0.491	0.595	0.484	0.639	0.596	-76.885	-164256.016	-83.542	0.501	0.472
HV with 30 data	0.855	0.834	0.854	0.864	0.825	-2.887	-22.242	-1.819	0.786	0.557

are

$$\begin{cases} \mu_1(\mathbf{x}_*) = \mathbf{k}_*^T \mathbf{K}_n^{-1} \mathbf{y} \\ \sigma_1^2(\mathbf{x}_*) = k(\mathbf{x}_*, \mathbf{x}_*) - \mathbf{k}_*^T \mathbf{K}_n^{-1} \mathbf{k}_* \end{cases} \quad (10)$$

where $k(x, y) = \frac{\sigma^2(2/\pi) \operatorname{asin}([\sigma_w^2 x^\top y + \sigma_b^2] / (\sqrt{\sigma_w^2 x^\top x + \sigma_b^2} + 1 \sqrt{\sigma_w^2 y^\top y + \sigma_b^2} + 1))}{\sigma^2(2/\pi)}$ in GP2 model, which is a MLP kernel (also known as arc sine kernel). The formulas for the RF model are

$$\begin{cases} \mu_2(\mathbf{x}_*) = \frac{1}{N} \sum_{i=1}^N T_i^* \\ \sigma_2^2(\mathbf{x}_*) = \frac{1}{N} \sum_{i=1}^N (T_i^* - \mu_2(\mathbf{x}_*))^2 \end{cases} \quad (11)$$

where T_i^* represents the prediction of the i th tree and N denotes the number of trees. The formulas for SVIDKL are

$$\begin{cases} \mu_3(\mathbf{x}^*) = \mathbf{k}_{\mathbf{x}^* \mathbf{Z}} (\mathbf{K}_{\mathbf{ZZ}} + \Sigma)^{-1} \mathbf{m} \\ \sigma_3^2(\mathbf{x}^*) = k_{\mathbf{x}^* \mathbf{x}^*} - \mathbf{k}_{\mathbf{x}^* \mathbf{Z}} (\mathbf{K}_{\mathbf{ZZ}} + \Sigma)^{-1} \mathbf{k}_{\mathbf{Z} \mathbf{x}^*} \end{cases} \quad (12)$$

where $\mathbf{k}_{\mathbf{x}^* \mathbf{Z}}$ is the covariance vector between the input $\mathbf{x}^*$ and all inducing points. $\mathbf{K}_{\mathbf{ZZ}}$ is the covariance matrix between the inducing points. $\mathbf{m}$ and Σ are the mean vector and covariance matrix of the variational distribution $q(\mathbf{u})$, respectively. $k_{\mathbf{x}^* \mathbf{x}^*}$ is the covariance of the input $\mathbf{x}^*$ itself.

The three base models are assumed to follow independent normal distributions, respectively, which are mathematically formulated as $\mathcal{N}(\mu_1, \sigma_1^2)$, $\mathcal{N}(\mu_2, \sigma_2^2)$, and $\mathcal{N}(\mu_3, \sigma_3^2)$. Predictions are randomly sampled from the three independent normal distributions, and an ensemble is created by calculating the average of the sampled predictions. This is because the predictions are used to calculate the EHVI acquisition function value, in which the Monte Carlo sampling method [43] is employed to obtain the expectation. Mathematically, the ensembled sampled prediction can be formulated as

$$\text{Sample} = (\text{Sample}_1 + \text{Sample}_2 + \text{Sample}_3) / 3. \quad (13)$$

E. Optimize EHVI Acquisition Function With Traversal Approach

In BO, acquisition functions play a crucial role in guiding the global optimization process by utilizing predictions from surrogate models. The EHVI function is essential in BO for multiobjective problems. It measures the expected increase in the hypervolume of the objective space that a new solution contributes, balancing exploration and exploitation. By using probabilistic models like GP, EHVI identifies candidate solutions that can significantly improve and diversify the current set of optimal solutions, making it effective for optimizing complex, multiobjective problems.

The hypervolume metric in EHVI is widely used to assess the quality of the Pareto-optimal set. It measures the volume of the objective space that is dominated by the Pareto front set

and the reference point. The definition of hypervolume in [44] is

$$\text{HV}_{\mathbf{y}_{\text{ref}}}(\mathcal{P}(\mathcal{Y})) = \int_{\mathcal{Y}} \mathbb{I}[\mathbf{y} \succ \mathbf{y}_{\text{ref}}] \left[1 - \prod_{\mathbf{y}_* \in \mathcal{P}(\mathcal{Y})} \mathbb{I}[\mathbf{y}_* \not\succ \mathbf{y}] \right] d\mathbf{y} \quad (14)$$

where $\mathcal{P}(\mathcal{Y})$ is the Pareto-optimal set and $\mathbf{y}_{\text{ref}}$ denotes the reference point which is dominated by each element of $\mathcal{P}(\mathcal{Y})$. The indicator function, denoted by $\mathbb{I}(\cdot)$, returns 1 if the input is true and 0 otherwise.

In Prior-boosted GRL, we employ an EHVI function [44] to measure the possible contribution of the design to the current Pareto front, which can be expressed as

$$\text{EHVI}(\mathbf{x}) = \mathbb{E}_{p(f(\mathbf{x}))} [\text{HV}_{\mathbf{y}_{\text{ref}}}(\mathcal{P}(\mathbf{y}) \cup f(\mathbf{x})) - \text{HV}_{\mathbf{y}_{\text{ref}}}(\mathcal{P}(\mathbf{y}))] \quad (15)$$

where $f(\mathbf{x})$ means the prediction of $\mathbf{x}$ from ensemble surrogate models. $\mathbb{E}_{p(f(\mathbf{x}))}(\cdot)$ outputs the expectation of the argument, which is based on the *Monte Carlo method*. We sample n predictions from the ensemble probabilistic model and calculate the average of the hypervolume improvement values of the samples as the expectation value.

Given the interdependence of microarchitecture parameters and the discrete nature of the search space, we catalog all valid designs within the ICCAD Contest 2022 design space. Subsequently, we employ *traversal techniques* to optimize the EHVI function. This approach effectively navigates the intricacies of architecture parameter interdependencies without incurring significant computational costs. The design with the highest-EHVI function value is deemed to possess the greatest potential to be Pareto-optimal and is thus selected as the suggested design for the VLSI evaluation flow. After evaluation, it will be added to the observed data set, and the optimization proceeds to the next iteration.

The EHVI acquisition function balances the exploration of negatively correlated PPA values, guiding the exploration in the vast design space. With the EHVI function, we can identify the most potential designs that contribute to the Pareto front and undergo time-consuming evaluation. It serves as a prudent guide that navigates the optimization toward the Pareto-optimal set with the fewest iterations, thereby enhancing the exploration efficiency.

IV. EXPERIMENTAL RESULTS

A. Experiment Setting

The Prior-boosted GRL algorithm is implemented in the Python programming language with the HEBO framework [45], which is a black-box optimization framework.

TABLE IV
COEFFICIENT OF DETERMINATION (R^2) VALUES OF THE PREDICTED PPA

	Performance	Power	Area
R^2	0.979376	0.995225	0.993132

HEBO offers various probabilistic surrogate models and diverse acquisition functions.

ICCAD Contest 2022 Problem C does not provide the corresponding PPA values of the microarchitecture design space in Table I. Therefore, we utilize RTL-level simulation tools and various RISC-V processor benchmarks to obtain the PPA data within the search space. The evaluation tools include the Chipyard framework [46], Synopsys VCS simulator, and Design Compiler. The simulations are executed on a cluster equipped with an Intel Xeon Platinum 8354H CPU@3.10 GHz. To evaluate the performance of the design, multiple CPU benchmarks are utilized, including dhrystone, median, mt-matmul, mt-vvadd, multiply, pmp, qsort, rsort, spmv, towers, vvadd, and mm. These benchmarks encompass a comprehensive set of RISC-V instructions, enabling a thorough assessment of the microarchitecture design's performance.

Due to the time-consuming nature of the VLSI verification flow, we obtain a limited amount of power and area data (896 samples) and performance data (1017 samples). We use machine learning models fitted by the existing data to acquire PPA values covering the entire search space. The xgboost model (XGB) [47] is chosen for its strong modeling capabilities and ease of training. CV results in Table IV demonstrate the XGB's effectiveness in capturing the BOOM processor's relationship between microarchitecture parameters and PPA values. The XGB's predictions of the entire design space, incorporating the Gaussian noise to mimic real-world scenarios, constitute a comprehensive dataset for evaluating the DSE algorithms.

To validate different DSE algorithms, we use the DSE algorithm benchmarking platform provided by the ICCAD Contest 2022 [17], which is an estimation algorithm written in Python. The evaluation criterion for DSE algorithms is the Pareto Hypervolume of the z -score normalized PPA, calculated according to (14). The reference point for calculating the Pareto Hypervolume is set as $[\min z_{\text{perf}}, \max z_{\text{power}}, \max z_{\text{area}}]$.

The presented experimental results are the average values obtained from ten individual experiments, ensuring robustness and reliability in evaluating the algorithms.

B. Graph Embedding Space Evaluation

1) *Graph Neural Network Training*: We generate a dataset of approximately 15 000 DAGs from the design search space of ICCAD Contest 2022 in Table I, with each DAG representing a distinct BOOM configuration in terms of microarchitecture. Unlike typical supervised learning, our GNN training does not necessitate time-consuming VLSI implementation flows. Utilizing the teacher forcing method [21], we effectively measure the reconstruction loss, following the approach in [24]. We employ mini-batch gradient

TABLE V
NORMALIZED DISTANCE COMPARISON IN PARAMETER VECTOR SPACE AND GRAPH EMBEDDING SPACE

Design space	Parameter vector space		Graph embedding space	
	x_1, x_2	x_1, x_3	z_1, z_2	z_1, z_3
Normalized distance	0.161	0.839	0.087	0.913

TABLE VI
COMPARISON OF INITIAL SAMPLING RESULTS

Algorithm	Hypervolume
10 random sampling points	13.7479
5 random sampling points + 5 optimization	15.9783
10 MicroAL sampling points	15.3534
5 MicroAL sampling points + 5 optimization	14.5646
10 PriorBoost Sampling points	16.4263

descent to optimize the DVAE loss function. The entire training process, conducted on a GeForce RTX 2080 Ti GPU, is completed in about 10 h, spanning 500 epochs. Notably, this training process is a one-time activity; the trained model can be reapplied for various BOOM optimizations, including those targeting diverse multiobjective design specifications, such as reliability and manufacturability.

2) *Evaluating the Quality of Graph Embedding Space*: To assess the effectiveness of the graph embedding space, we execute a reverse mapping experiment. We randomly select three points in the PPA objective space, denoted as y_1 , y_2 (close to y_1), and y_3 . These points are then mapped back to both the original parameter vector space (x_1, x_2, x_3) and the graph embedding space (z_1, z_2, z_3). The results in Table V shows that. The normalized distances between these points are calculated and averaged over 10 iterations. Notably, the distance between x_1 and x_2 is found to be 85.06% larger than the distance between z_1 and z_2 , which demonstrates the continuity and effectiveness of the proposed graph embedding space.

C. Efficiency of the Initial Sampling Algorithm

The PriorBoost sampling algorithm can leverage prior knowledge, thereby generating high-quality designs that are closely proximity to the PPA Pareto frontier. Table VI illustrates the Pareto hypervolume achieved by PriorBoost Sampling, as well as by other DSE sampling algorithms. PriorBoost Sampling outperforms MicroAL [14], the initial sampling algorithm employed by the DSE algorithm BOOM-Explorer [14], and the widely used random sampling algorithm within the same sampling designs. Compared to the result of MicroAL and random sampling coupled with five rounds of optimization, PriorBoost Sampling still retains the best-hypervolume value. The optimization result of MicroAL underperforms its direct sampling result, possibly due to the scarcity of initial samples, which results in a less accurate surrogate model.

D. Comparison With Prior Work and Official Configuration

The performance of our DSE framework is compared with the top-performing BOOM DSE algorithm, BOOM-Explorer [14], which is a state-of-the-art BOOM DSE algorithm. We also compare Prior-boosted GRL with its

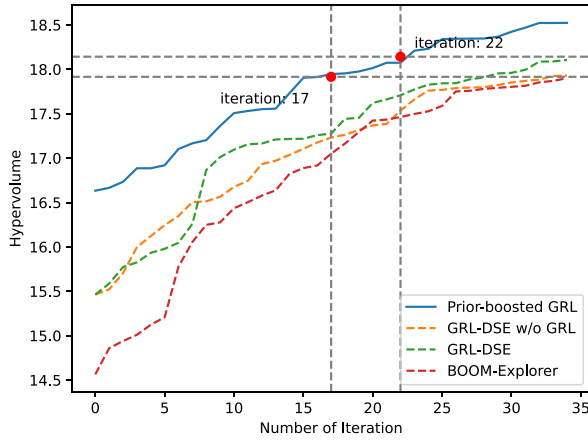


Fig. 5. Comparison of iteration curves of several outstanding DSE algorithms.

TABLE VII
COMPARISON OF THE QUALITY OF THE SEARCHED DESIGNS

Algorithm	Prior-boosted GRL	GRL-DSE [15]	BOOM-Explorer [14]
Hypervolume	18.2543	18.0343	17.9432
Golden Distance	0.0809	0.1393	0.1681

previous version in [15]. In [14], BOOM-Explorer only chooses five initial sampling points, while Prior-boosted GRL and GRL-DSE select ten designs as the initial sampling set, respectively. To maintain an equal number of evaluations, we provide 5 more rounds of optimization iteration for the BOOM-Explorer at the starting point. The experimental results are shown in Fig. 5.

From the optimization results in Fig. 5, it can be observed that Prior-boosted GRL, compared to the current state-of-the-art BOOM-Explorer algorithm, exhibits superior microarchitecture optimization effects. This highlights the advantage of embedding microarchitecture dependencies into the search space and utilizing graph learning for mapping before searching. We also compare the experimental results without using graph learning, revealing a significant decrease in search effectiveness. The Prior-boosted GRL algorithm, incorporating the PriorBoost initial sampling method and an enhanced ensemble model, demonstrates superior performance in the optimization experiments. It is evident that the Prior-boosted GRL framework exhibits a significant advantage from the beginning and maintains a smoother improvement throughout the optimization process, ultimately achieving the best results in DSE within the same number of iterations.

According to our analysis of the optimization process, Prior-boosted GRL can achieve the hypervolume of GRL-DSE with 35 rounds of optimizations in merely 22 rounds of optimizations. This leads to a 28% reduction in exploration time and significantly enhances the efficiency of DSE. Compared to BOOM-Explorer, Prior-boosted GRL can achieve a similar hypervolume of BOOM-Explorer with 40 rounds of iterations with just 17 iterations, thereby reducing the number of iterations by 57.5%.

We conduct optimization experiments by combining different base learners and weights in various combinations. The results of these experiments, as depicted in Fig. 6, indicate that the combination of the RF model, SVIDKL model, and GP model outperforms other combinations.

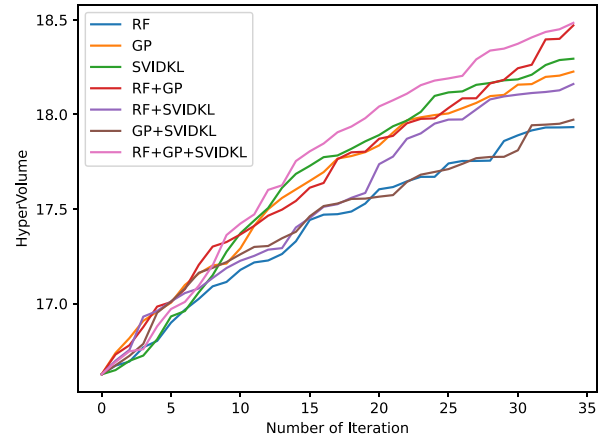
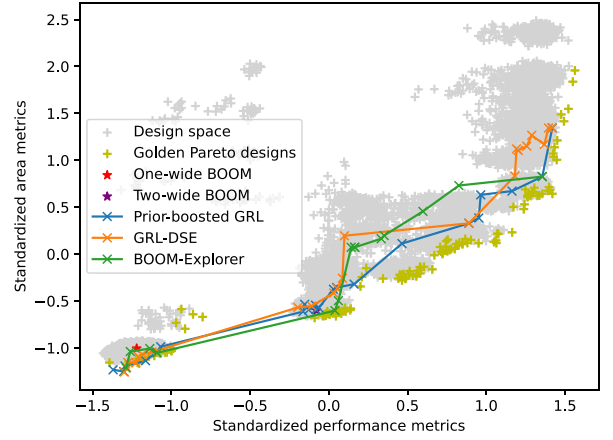
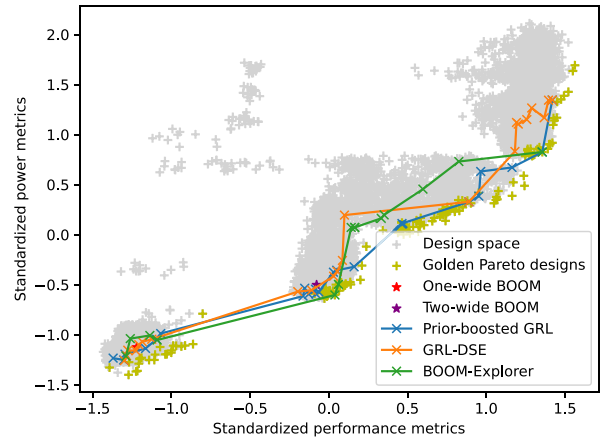


Fig. 6. Comparison of optimization results of various ensemble models. *RF* denotes the RF regression model, *GP* denotes the GP model, and *FE* denotes the feature selection ensemble model.



(a)



(b)

Fig. 7. Pareto frontier comparison among various DSE algorithms. (a) Pareto frontier in performance-area space. (b) Pareto frontier in performance-power space.

Generational distance (GD) [48] measures the distance from the solution to the golden Pareto front and is utilized to evaluate the quality of the searched designs in the DSE

TABLE VIII
ABLATION EXPERIMENT RESULT COMPARED TO BOOM-EXPLORER [14]

	①Fine-tuned Ensemble Model	②Graph Learning	③Prior-boost Sampling	HV	HV Improvement Ratio
BOOM-Explorer [14]				17.9432	0%
With ①	✓			18.0323	28.62%
With ①&②	✓	✓		18.1801	76.13%
Prior-boosted GRL	✓	✓	✓	18.2543	100%

TABLE IX
COMPARISON BETWEEN PRIOR-BOOSTED GRL PARETO-OPTIMAL DESIGNS AND THE BOOM OFFICIAL DESIGN

	p	p	g	g
Microarchitecture Design	Configure Parameters	Performance(IPC)↑ Value Ratio	Power(W)↓ Value Ratio	Area(μm^2)↓ Value Ratio
One-wide BOOM	[4, 1, 1, 1, 8, 1, 1, 8, 1, 1, 8, 8, 16, 32, 52, 48, 8, 8, 64, 4, 1, 32, 64, 4, 0, 2, 1, 8]	0.7140	2.8714	3.2014 × 10 ⁴
GRL-DSE	[4, 1, 1, 1, 6, 1, 1, 6, 1, 1, 6, 8, 16, 34, 52, 48, 12, 12, 32, 4, 1, 16, 64, 4, 1, 8, 2, 1]	0.7280	2.832	3.1002 × 10 ⁴
Prior-boosted GRL	[4, 1, 1, 1, 6, 1, 1, 6, 1, 1, 6, 6, 14, 34, 52, 48, 12, 12, 64, 1, 1, 16, 64, 4, 1, 2, 1, 32]	0.7419	2.831	3.0992 × 10⁴
Two-wide BOOM	[4, 2, 2, 1, 12, 2, 2, 20, 2, 1, 16, 12, 16, 32, 64, 80, 64, 16, 16, 64, 4, 1, 32, 64, 4, 0, 2, 1, 8]	1.015	4.033	4.5211 × 10 ⁴
GRL-DSE	[4, 2, 2, 1, 12, 2, 2, 20, 2, 1, 16, 12, 16, 32, 60, 80, 64, 16, 16, 32, 4, 1, 16, 64, 4, 1, 8, 2, 0]	1.016	4.002	4.4833 × 10 ⁴
Prior-boosted GRL	[4, 2, 2, 1, 12, 2, 2, 20, 2, 1, 16, 12, 16, 32, 60, 70, 54, 20, 20, 32, 4, 1, 16, 64, 4, 1, 2, 1, 8]	1.0541	3.8414	4.4421 × 10⁴

TABLE X
COMPARISON OF ONLINE DSE ALGORITHM ESTIMATION PLATFORM RESULTS

Algorithm	3rd Place	2nd Place	1st Place	Prior-boosted GRL	GRL-DSE [16]	cad1021	cad1013	cad1012
Hypervolume	-	-	-	7.9071	7.8454	7.9002	7.6899	7.7825
Running Time	-	-	-	8.9387 × 10⁵	9.0307 × 10 ⁵	1.0868 × 10 ⁶	9.4033 × 10 ⁵	1.1840 × 10 ⁶
Score	51.1285	51.4003	52.0922	52.6571	52.2189	52.0303	51.0748	50.9672

problem. The definition of GD is shown below

$$GD(\Gamma, \Omega) = \frac{1}{|\Gamma|} \sum_{\gamma \in \Gamma} \min_{\omega \in \Omega} L(\gamma, \omega) \quad (16)$$

where Γ is the Pareto front searched by the algorithm, and Ω is the golden Pareto front. The golden Pareto front is the optimal Pareto front identified within the dataset predicted by the XGB. The results in Table VII show that Prior-boosted GRL outperforms the GRL-DSE by 72.19% and BOOM-Explorer by 107.79%.

To verify the effectiveness of the three main contributions in our proposed method, we conducted ablation experiments. The experiment results compared to BOOM-Explorer [14] are shown in Table VIII. The contributions of fine-tuned ensemble model, graph learning, fine-tuned ensemble model, and prior-boost sampling to the searched results compared to BOOM-Explorer are 28.62%, 47.51%, and 23.87%, respectively. The ensemble model offers an accurate and robust prediction for the BO. The graph learning method encodes the microarchitecture interdependence information to present a high-quality search space. Additionally, the Prior-boost sampling algorithm further improves the quality of the initial sampling, helping to achieve better results.

E. Comparison of Pareto-Optimal BOOM Microarchitectures

In Fig. 7, we mark the Pareto frontiers found by the DSE algorithms in performance-power and performance-area space. By comparing the Pareto front of Prior-boosted GRL with that of GRL-DSE and BOOM-Explorer, we find that the Pareto front of Prior-boosted GRL is closer to the golden Pareto front. Additionally, Prior-boosted GRL finds three designs on the golden Pareto front, while GRL-DSE and BOOM-Explorer find 2 and 0 designs, respectively. This suggests that, given an equal number of searches, Prior-boosted GRL demonstrates superior performance and enhanced efficiency.

Several designs selected by Prior-boosted GRL have been evaluated using the VLSI verification flow and compared

with the official BOOM design configurations [49]. Their PPA metrics are displayed in Table IX, where the unit of performance is denoted in instructions per cycle (IPC). The results demonstrate that, due to careful consideration of dependencies between architectures during the optimization process, GRL-DSE outperforms the officially provided one-wide and two-wide BOOM processor microarchitectures. The Prior-boosted GRL further enhances the initial sampling algorithm and surrogate model, enabling the exploration of better-processor microarchitectures. Compared to the official designs, Prior-boosted GRL achieves a performance improvement of 1% to 5% on the PPA metrics.

F. Results of the ICCAD Contest DSE Ranking System

In addition to validating our algorithm on local datasets, we also submit the algorithm to the online BOOM DSE algorithm validation platform of ICCAD Contest 2022 [50]. It not only employs a search space containing parameters of Table I but also has potential hidden design configurations. Furthermore, the exact PPA parameters remain undisclosed. The platform accepts DSE algorithms and returns Pareto hypervolume, overall running time (ORT), and final score. The runtime value includes both algorithm execution time and simulation time. Equation (17) claims the calculation of the score. α is the baseline of the ORT, which equals 6. θ is the predefined time budget, which equals 2625000. Higher hypervolume and lower-runtime ORT lead to a higher score

$$\text{score} = \text{Hypervolume} \times \begin{cases} \alpha - \frac{\text{ORT} - \theta}{\theta}, & \text{ORT} \geq \theta \\ \alpha + \left| \frac{\text{ORT} - \theta}{\theta} \right|, & \text{ORT} < \theta. \end{cases} \quad (17)$$

The hypervolume, time, and scores of different teams and algorithms are listed in Table X. First, upon comparing prior-boosted GRL with the victors of the 2022 ICCAD Contest Problem C, denoted 1st, 2nd, and 3rd Place, it is evident that Prior-boosted GRL noticeably surpasses them

in terms of scores. As the committee only disclosed the winners' scores, we analyzed data from the ranking list during the contest [50], where the names of the top three teams are cadc1021, cadc1013, and cadc1012, respectively. The results show that Prior-boosted GRL achieves a slightly higher hypervolume than the top-1 team cadc1021 within a 17.7% shorter runtime, placing our score far ahead. When compared to the score of the previous version of GRL-DSE [15], Prior-boosted GRL attains a higher hypervolume in less time, conclusively demonstrating Prior-boosted GRL's capability to enhance DSE efficiency greatly. By the submission deadline of this article, Prior-boosted GRL retains the top position in the ranking list.

V. CONCLUSION

This article introduces Prior-boosted GRL, an automated microarchitecture search framework that leverages GRL to capture microarchitecture knowledge. It incorporates processor architecture knowledge into a graph representation space and utilizes efficient initial sampling along with an ensemble learning-based multiobjective BO algorithm to explore the optimal microarchitecture designs. The experimental studies demonstrate that Prior-boosted GRL surpasses the state-of-the-art DSE algorithms in terms of Pareto-optimal quality. The designs yielded by Prior-boosted GRL outperform manual designs of BOOM. Moreover, Prior-boosted GRL achieves superior performance compared to the winners of the ICCAD CAD Contest 2022.

REFERENCES

- [1] N. Binkert et al., "The gem5 simulator," *ACM SIGARCH Comput. Archit. News*, vol. 39, no. 2, pp. 1–7, 2011.
- [2] S. Li, J. H. Ahn, R. D. Strong, J. B. Brockman, D. M. Tullsen, and N. P. Jouppi, "McPAT: An integrated power, area, and timing modeling framework for multicore and manycore architectures," in *Proc. 42nd Annu. IEEE/ACM Int. Symp. Microarchit.*, 2009, pp. 469–480.
- [3] A. Roelke and M. R. Stan, "RISC5: Implementing the RISC-V ISA in gem5," in *Proc. 1st Workshop Comput. Archit. Res. RISC-V (CARRV)*, 2017, pp. 1–7.
- [4] E. Perelman, G. Hamerly, M. Van Biesbrouck, T. Sherwood, and B. Calder, "Using SimPoint for accurate and efficient simulation," *ACM SIGMETRICS Perform. Eval. Rev.*, vol. 31, no. 1, pp. 318–319, 2003.
- [5] R. E. Wunderlich, T. F. Wenisch, B. Falsafi, and J. C. Hoe, "SMARTS: Accelerating microarchitecture simulation via rigorous statistical sampling," in *Proc. 30th Annu. Int. Symp. Comput. Archit.*, 2003, pp. 84–95.
- [6] T. Austin, E. Larson, and D. Ernst, "SimpleScalar: An infrastructure for computer system modeling," *Computer*, vol. 35, no. 2, pp. 59–67, Feb. 2002.
- [7] Z. Wu, X. Yi, L. Shang, and F. Yang, "SenseDSE: Sensitivity-based performance evaluation for design space exploration of microarchitecture," in *Proc. Design, Autom. Test Eur. Conf. Exhib. (DATE)*, 2024, pp. 1–6.
- [8] E. İpek, S. A. McKee, R. Caruana, B. R. de Supinski, and M. Schulz, "Efficiently exploring architectural design spaces via predictive modeling," *ACM SIGOPS Oper. Syst. Rev.*, vol. 40, no. 5, pp. 195–206, 2006.
- [9] D. Li, S. Yao, Y.-H. Liu, S. Wang, and X.-H. Sun, "Efficient design space exploration via statistical sampling and AdaBoost learning," in *Proc. 53rd Annu. Design Autom. Conf.*, 2016, pp. 1–6.
- [10] B. C. Lee and D. M. Brooks, "Illustrative design space studies with microarchitectural regression models," in *Proc. IEEE 13th Int. Symp. High Perform. Comput. Archit.*, 2007, pp. 340–351.
- [11] S. Khan, P. Xekalakis, J. Cavazos, and M. Cintra, "Using PredictiveModeling for cross-program design space exploration in multicore systems," in *Proc. 16th Int. Conf. Parallel Archit. Compil. Techn. (PACT)*, 2007, pp. 327–338.
- [12] P. J. Joseph, K. Vaswani, and M. J. Thazhuthaveetil, "Construction and use of linear regression models for processor performance analysis," in *Proc. 12th Int. Symp. High-Perform. Comput. Archit.*, 2006, pp. 99–108.
- [13] G. Beltrame, L. Fossati, and D. Sciuto, "Decision-theoretic design space exploration of multiprocessor platforms," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 29, no. 7, pp. 1083–1095, Jul. 2010.
- [14] C. Bai, Q. Sun, J. Zhai, Y. Ma, B. Yu, and M. D. F. Wong, "BOOM-explorer: RISC-V BOOM microarchitecture design space exploration framework," in *Proc. IEEE/ACM Int. Conf. Comput. Aided Design (ICCAD)*, 2021, pp. 1–9.
- [15] X. Yi, J. Lu, X. Xiong, D. Xu, L. Shang, and F. Yang, "Graph representation learning for microarchitecture design space exploration," in *Proc. IEEE/ACM Design Autom. Conf. (DAC)*, 2023, pp. 1–6.
- [16] K. Asanovic, D. A. Patterson, and C. Celio, "The Berkeley out-of-order machine (BOOM): An industry-competitive, synthesizable, parameterized RISC-V processor," EECS Dept., Univ. California, Berkeley, CA, USA, Rep. UCB/EECS-2015-167, 2015.
- [17] X. W. S. Li and C. Bai, "ICCAD CAD contest 2022." 2022. [Online]. Available: <https://dl.acm.org/doi/abs/10.1145/3508352.3561109>
- [18] J. Bachrach et al., "Chisel: Constructing hardware in a scala embedded language," in *Proc. Design Autom. Conf. (DAC)*, 2012, pp. 1212–1221.
- [19] H. Cai, V. W. Zheng, and K. C.-C. Chang, "A comprehensive survey of graph embedding: Problems, techniques, and applications," *IEEE Trans. Knowl. Data Eng.*, vol. 30, no. 9, pp. 1616–1637, Sep. 2018.
- [20] F. Scarselli, M. Gori, A. C. Tsoi, M. Hagenbuchner, and G. Monfardini, "The graph neural network model," *IEEE Trans. Neural Netw.*, vol. 20, no. 1, pp. 61–80, Jan. 2009.
- [21] W. Jin, R. Barzilay, and T. Jaakkola, "Junction tree variational autoencoder for molecular graph generation," in *Proc. 35th Int. Conf. Mach. Learn.*, 2018, pp. 2323–2332.
- [22] Z. Wu, S. Pan, F. Chen, G. Long, C. Zhang, and S. Y. Philip, "A comprehensive survey on graph neural networks," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 32, no. 1, pp. 4–24, Jan. 2021.
- [23] T. N. Kipf and M. Welling, "Variational graph auto-encoders," 2016, *arXiv:1611.07308*.
- [24] M. Zhang, S. Jiang, Z. Cui, R. Garnett, and Y. Chen, "D-VAE: A variational autoencoder for directed acyclic graphs," in *Proc. 33rd Adv. Neural Inf. Process. Syst.*, 2019, pp. 1–13.
- [25] R. Singh, N. S. Mangat, R. Singh, and N. S. Mangat, "Stratified sampling," in *Elements of Survey Sampling*. Dordrecht, The Netherlands: Springer, 1996, pp. 102–144.
- [26] S. T. Tokdar and R. E. Kass, "Importance sampling: A review," *Wiley Interdiscip. Rev. Comput. Statist.*, vol. 2, no. 1, pp. 54–60, 2010.
- [27] G. Hinton, N. Srivastava, and K. Swersky, "Neural networks for machine learning lecture 6a overview of mini-batch gradient descent," *Cited*, vol. 14, no. 8, p. 2, 2012.
- [28] K. Gallagher, K. Charvin, S. Nielsen, M. Sambridge, and J. Stephenson, "Markov chain monte carlo (MCMC) sampling methods to determine optimal models, model resolution and model choice for earth science problems," *Mar. Petroleum Geol.*, vol. 26, no. 4, pp. 525–535, 2009.
- [29] A. E. Gelfand, "Gibbs sampling," *J. Am. Stat. Assoc.*, vol. 95, no. 452, pp. 1300–1304, 2000.
- [30] J. Mockus, *Bayesian Approach to Global Optimization: Theory and Applications*, vol. 37, Dordrecht, The Netherlands: Springer, 2012.
- [31] S. Eyerman, L. Eeckhout, T. Karkhanis, and J. E. Smith, "A mechanistic performance model for superscalar out-of-order processors," *ACM Trans. Comput. Syst.*, vol. 27, no. 2, pp. 1–37, 2009.
- [32] K. Cho et al., "Learning phrase representations using RNN encoder-decoder for statistical machine translation," 2014, *arXiv:1406.1078*.
- [33] M. Ahmed, R. Seraj, and S. M. S. Islam, "The k-means algorithm: A comprehensive survey and performance evaluation," *Electronics*, vol. 9, no. 8, p. 1295, 2020.
- [34] F. Hutter, H. H. Hoos, and K. Leyton-Brown, "Sequential model-based optimization for general algorithm configuration," in *Proc. 5th Int. Conf. Learn. Intell. Optim.*, 2011, pp. 507–523.
- [35] C. E. Rasmussen, "Gaussian processes in machine learning," in *Advanced Lectures on Machine Learning* (Summer School on Machine Learning). Berlin, Germany: Springer, 2003, pp. 63–71.
- [36] B. Lakshminarayanan, A. Pritzel, and C. Blundell, "Simple and scalable predictive uncertainty estimation using deep ensembles," in *Proc. 31st Adv. Neural Inf. Process. Syst.*, 2017, pp. 1–12.
- [37] F. Leibfried, V. Dutordoir, S. John, and N. Durrande, "A tutorial on sparse gaussian processes and variational inference," 2022, *arXiv:2012.13962*.

- [38] A. G. Wilson, Z. Hu, R. R. Salakhutdinov, and E. P. Xing, "Stochastic variational deep kernel learning," in *Proc. Adv. Neural Inf. Process. Syst.*, 2016, pp. 1–9.
- [39] C. Li, C. Chen, D. Carlson, and L. Carin, "Preconditioned stochastic gradient Langevin dynamics for deep neural networks," in *Proc. 13th AAAI Conf. Artif. Intell.*, 2016, pp. 1–7.
- [40] M. Teye, H. Azizpour, and K. Smith, "Bayesian uncertainty estimation for batch normalized deep networks," in *Proc. 35th Int. Conf. Mach. Learn.*, 2018, pp. 4907–4916.
- [41] Y. Yamada, O. Lindenbaum, S. Negahban, and Y. Kluger, "Feature selection using stochastic gates," in *Proc. 37th Int. Conf. Mach. Learn.*, 2020, pp. 10648–10659.
- [42] E. Jang, S. Gu, and B. Poole, "Categorical reparameterization with Gumbel–Softmax," 2017, *arXiv:1611.01144*.
- [43] R. L. Harrison, "Introduction to monte carlo simulation," in *Proc. AIP Conf.*, 2010, pp. 17–21.
- [44] A. Shah and Z. Ghahramani, "Pareto frontier learning with expensive correlated objectives," in *Proc. 33rd Int. Conf. Mach. Learn.*, 2016, pp. 1919–1927.
- [45] A. I. Cowen-Rivers et al., "An empirical study of assumptions in Bayesian optimisation," 2021, *arXiv:2012.03826*.
- [46] A. Amid et al., "Chipyard: Integrated design, simulation, and implementation framework for custom SoCs," *IEEE Micro*, vol. 40, no. 4, pp. 10–21, Jul./Aug. 2020.
- [47] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowl. Discov. Data Min.*, 2016, pp. 785–794.
- [48] D. A. Van Veldhuizen, *Multiobjective Evolutionary Algorithms: Classifications, Analyses, and New Innovations*. Wright-Patterson AFB, OH, USA: Air Force Inst. Technol., 1999.
- [49] J. Zhao, B. Korpan, A. Gonzalez, and K. Asanovic, "SonicBOOM: The 3rd generation Berkeley out-of-order machine," in *Proc. 4th Workshop Comput. Archit. Res. RISC-V*, 2020, pp. 1–7.
- [50] "2022 CAD contest @ ICCAD—Problem C: CAD contest problem C ranking online platform." Aug. 2022. [Online]. Available: <http://47.93.191.38/>



Zheng Wu received the B.E. degree in integrated circuit design and integrated system from the Huazhong University of Science and Technology, Wuhan, Hubei, China, in 2020. He is currently pursuing the Ph.D. degree with the State Key Laboratory of Integrated Chips and System, Microelectronics Department, Fudan University, Shanghai, China.

His current research interests include processor architecture design space exploration and automated deep learning compilation.



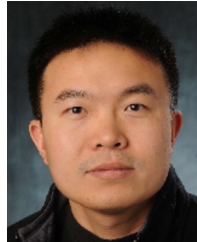
Jinyi Shen received the B.S. degree in microelectronics from Fudan University, Shanghai, China, in 2021, where he is currently pursuing the Ph.D. degree with the School of Microelectronics.

His current research interests include analog circuit design automation and optimization, and microarchitecture design space exploration.



Xiaoling Yi received the B.Sc. degree in microelectronics science and engineering from Fudan University, Shanghai, China, in 2020, and the M.Sc. degree in integrated circuit and system design from Fudan University, in 2020 and 2023, respectively. She is currently pursuing the Ph.D. degree in energy-efficient multicore heterogeneous systems for embedded machine learning from the Microelectronics Circuits and Systems Laboratories, KU Leuven, Leuven, Belgium.

Her current research interests include ML accelerators, reduced instruction set computer-V architecture, and heterogeneous multicore systems.



Li Shang (Member, IEEE) received the Ph.D. degree from Princeton University, Princeton, NJ, USA, in 2004.

He was the Deputy Director and the Chief Architect of the Intel Labs, Beijing, China, and an Associate Professor of the CU Boulder, Boulder, CO, USA. He is a Professor with the School of Computer Science, Fudan University, Shanghai, China. His research interests include computer systems, human-centered computing, machine learning, VLSI, and EDA, with over 100 publications, multiple best paper awards and nominations.

Prof. Shang was a recipient of the NSF Career Award.



Fan Yang (Member, IEEE) received the B.S. degree from Xi'an Jiaotong University, Xi'an, China, in 2003, and the Ph.D. degree from Fudan University, Shanghai, China, in 2008.

He is currently a Full Professor with the Microelectronics Department, Fudan University. His research interests include model order reduction, circuit simulation, high-level synthesis, acceleration of artificial neural networks, and yield analysis and design for manufacturability.



Xuan Zeng (Senior Member, IEEE) received the B.S. and Ph.D. degrees in electrical engineering from Fudan University, Shanghai, China, in 1991 and 1997, respectively.

She is a Full Professor with the Department of Microelectronics, Fudan University. She was the Director of the State Key Laboratory of Integrated Chips and System, Microelectronics Department, Fudan University, Shanghai, China, from 2008 to 2012. She was a Visiting Professor with the Department of Electrical Engineering, Texas

A&M University, College Station, TX, USA, and the Department of Microelectronics, Technische Universiteit Delft, Delft, The Netherlands, in 2002 and 2003, respectively.